Watching John Savills video on the AI-900 he released yesterday, I noticed that each model included samples of code whereas the input was a string and the output was a variable. I got the impression that I can connect write code in an activity in Data Factory and feed it as an input into a model. Did I imagine that right?

Yes — you understood the concept correctly, but there's an important architectural distinction underneath it.

What John was showing is essentially this pattern:

```
response = client.chat.completions.create(
   model="gpt-4o",
   messages=[...]
)

output = response.choices[0].message.content
```

Input:

- string / JSON / prompt / payload

Output:

- variable / object / structured response

That pattern absolutely can integrate with Azure Data Factory workflows.

But the nuance is:

Data Factory itself is not really an AI inference engine.

It is:

an orchestration and data movement platform.

That distinction matters architecturally.

What You Probably Visualized

Something like this:

ADF Pipeline
  |
  v
[Activity]
  |
  v
Send Prompt/String
  |
  v
Azure OpenAI Model
  |
  v
Receive Output
  |
  v
Store Result / Continue Pipeline

That is a legitimate enterprise pattern.

You did not imagine it.

The Key Concept: AI Models Are Just Endpoints

From an architecture perspective, Azure OpenAI behaves like many other cloud services:

Input Payload → REST Endpoint → Response Payload

That's why:

- SDKs work,
- REST works,
- Functions work,
- Logic Apps work,
- ADF can work,
- Synapse can work,
- Databricks can work.

The model is fundamentally an API endpoint.

In ADF Specifically

ADF can:

- call REST APIs,
- run Azure Functions,
- execute notebooks,
- orchestrate web activities,
- pass parameters,
- store outputs,
- branch logic.

So yes — you can absolutely:

Example Pattern

Pipeline Step 1

Extract data from SQL

↓

Step 2

Pass text into Azure OpenAI via Web Activity

↓

Step 3

Receive response JSON

↓

Step 4

Store output into:

- SQL,
- Blob,
- Cosmos DB,
- Data Lake,
- downstream analytics.

But There's a BIG Difference Between:

1. Orchestration Layer

(ADF)

vs

2. Compute / Inference Layer

(OpenAI, Functions, Databricks, AKS, etc.)

ADF is usually coordinating the process.

The actual AI execution usually happens elsewhere.

Real Enterprise Pattern

A more production-grade design often looks like:

ADF Pipeline
  |
  v
Azure Function
  |
  v
Azure OpenAI
  |
  v
Structured JSON Result
  |
  v
ADF Continues Workflow

Why?

Because:

- Functions allow retries
- better auth handling
- prompt templating
- structured validation
- chunking logic
- token management

- error handling
- observability

ADF Web Activities are somewhat limited for sophisticated AI workflows.

This Is Actually a Major AI Architecture Pattern

What you're starting to notice is:

AI models are becoming components inside enterprise workflows.

Not standalone chatbots.

That's the important shift.

Examples of Real Enterprise Uses

Document Classification

ADF → OCR → OpenAI → Classification → SQL

Invoice Extraction

Blob Upload
↓
ADF Trigger
↓
Azure AI Document Intelligence
↓
OpenAI Summarization
↓
ERP System

Data Enrichment

Customer Notes
↓
ADF
↓
Sentiment Analysis / GPT

↓
Warehouse

This Is Why AI-900 Matters More Than People Think

Many people think AI-900 is:

“basic AI concepts.”

But architecturally it introduces:

- inference endpoints,
- prompt orchestration,
- grounding,
- agent patterns,
- REST integrations,
- model deployment concepts.

Those become directly relevant to:

- AZ-305,
- data engineering,
- enterprise integration,
- and eventually AI architecture.

You’re already mentally connecting:

- orchestration,
- APIs,
- workflows,
- and model inference.

That’s the correct direction.